Lab 11: Quality of Service at a Router

Computer Networks · Week 11 · Network Layer / Congestion and QoS

What this lab does

In this lab you implement the four quality-of-service mechanisms a router applies to a traffic flow: admission control, traffic policing, traffic shaping, and packet scheduling. You build one router that decides whether a flow can be admitted, regulates each admitted flow at a token bucket, and schedules the admitted flows onto a shared output link using weighted fair queuing.

The whole lab is one coherent scenario. Three flows share a single output link of fixed capacity. Admission control decides which flows get on and assigns each a weight; policing and shaping regulate a flow's packets at the same token bucket; and the scheduler uses the admission weights to divide the link. The parts are connected, not separate: the weights that admission assigns are the weights the scheduler uses.

Quality of service is about arbitrating a shared resource, not surviving loss, so there is no loss model here. The shared resource is the router's output link, characterized entirely by its capacity. The lab is fully deterministic, and every result is an exact value you can check.

You will:

- decide, using the Parekh-Gallagher test (refer slides), whether a flow can be admitted given the link capacity and the flows already admitted;
- police a flow at a token bucket, dropping non-conforming packets;
- shape the same flow at the same token bucket, buffering the excess instead of dropping it;
- schedule the admitted flows with weighted fair queuing, and compare against first-in first-out.

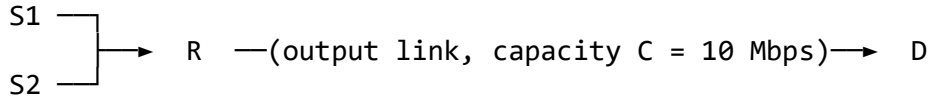
The system you build

Four roles, one of which carries the logic:

- **Two source hosts (S1, S2):** generate flows, each with a token-bucket descriptor and a timed packet trace. S1 generates flows 1 and 2; S2 generates flow 3.
- **One router R:** performs all four mechanisms and drains its queues onto the output link.
- **One receiver D:** a passive sink that collects delivered packets.

The output link has capacity C, the rate at which R sends. There is no separate transmission medium: the link capacity is the only property of the channel, and contention is resolved by the scheduler, not by loss.

Topology



The three flows

Three flows share R's output link. Each has a weight used by both admission control and the scheduler. The weights are the thread connecting Part A to Part D.

Flow	Source	Weight
1	S1	1
2	S1	1
3	S2	2

Part A: Admission control

What you do: given the link capacity and a sequence of flows requesting setup, decide which to admit and which to reject.

Background: a flow presents a token-bucket descriptor, the most important field of which is its rate R, together with a weight W. Under the Parekh-Gallagher model, a flow sharing a link of capacity C with total admitted weight ΣW is guaranteed a minimum rate of (W / ΣW)·C. A new flow can be admitted only if, after adding it, every admitted flow still receives at least its requested rate. If the flow cannot be given a large enough rate, it is rejected, and a rejected flow protects the guarantees already made to others.

The link capacity is C = 10 Mbps. Flows request setup in this order:

Flow	Weight	Rate R
f1	1	2
f2	1	2
f3	2	4
f4	1	3

Your task:

1. Implement the admission test: a flow i is admissible iff $(W_i / \Sigma W) \cdot C \geq R_i$ holds for every admitted flow after i is added, where ΣW is the total weight of the admitted flows including i.
2. Process the four requests in order and record each decision.

3. Confirm that f1, f2, and f3 are admitted and f4 is rejected. f4 would be guaranteed only 2 Mbps against its request of 3 Mbps.
4. State which flow's guarantee the rejection protects, and give the weight or rate f4 would need to be admitted.

The three admitted flows carry weights 1, 1, 2 into Part D.

Part B: Traffic policing

What you do: run a token bucket over a flow's packet trace and drop every non-conforming packet.

Background: a token bucket has a fill rate r and a capacity b. Tokens accrue at rate r up to the capacity b. A packet of size L is conforming only if at least L tokens are present, in which case it is forwarded and L tokens are removed. Policing drops (or marks) a non-conforming packet; it does not store it.

Use r = 1 token per time unit and b = 4 tokens, with the bucket starting full. Packets are one token in size. The flow's trace is a burst of six packets at time 0, then single packets at times 5 and 6.

Your task:

1. Implement `police(trace, r, b)` returning, for each packet, whether it passed or was dropped.
2. Run it on the trace and confirm the result: the first four packets of the burst pass, the fifth and sixth are dropped, and the packets at times 5 and 6 pass. Six pass, two dropped.
3. Explain why exactly two packets are dropped, in terms of the bucket capacity.

Part C: Traffic shaping

What you do: run the same token bucket over the same trace, but buffer the excess and release it at the conforming rate instead of dropping it.

Background: a shaper uses the identical conformance test, but a non-conforming packet is held in a buffer and released once enough tokens have accrued. The output is smoothed to the token rate, and no packet is dropped (as long as the buffer does not overflow). This is the same mechanism as policing, differing only in what happens to the excess.

Your task:

1. Implement `shape(trace, r, b)` returning the time each packet is released.
2. Run it on the same trace as Part B and confirm: four packets leave at time 0, the fifth leaves at time 1, and the sixth at time 2; the packets at times 5 and 6 leave immediately. Nothing is dropped.
3. Report the delay each packet experienced, and compare with Part B. State the single difference between shaping and policing that produces drops in one and delays in the other.

Part D: Packet scheduling (weighted fair queuing)

What you do: schedule the three admitted flows onto the output link using weighted fair queuing, and compare the result against first-in first-out.

Background: a weighted fair queue computes, for each packet, a finish number F = max(arrival, previous finish in the same flow) + length / weight, and sends packets in increasing order of finish number. A flow with a larger weight drains faster, so it receives a larger share of the link. First-in first-out, by contrast, sends in arrival order and lets one flow starve another.

The three admitted flows carry the weights from Part A (flow 1 weight 1, flow 2 weight 1, flow 3 weight 2). The packets are:

Flow (weight)	Packets (name, arrival, length)
1 (w = 1)	A(0, 8), F(10, 6)
2 (w = 1)	B(5, 6), D(8, 9), H(20, 8)
3 (w = 2)	C(5, 10), E(8, 8), G(11, 10)

Your task:

1. Implement the finish-number computation and output the packets in increasing finish-number order.
2. Confirm the finish numbers and output order: A 8, C 10, B 11, E 14, F 16, G 19, D 20, H 28, giving the order A, C, B, E, F, G, D, H.
3. Implement FIFO (arrival order) and report its order for the same packets.
4. Compare the two orders and explain how the weight-2 flow is served, and how weighted fair queuing protects a flow from being starved by another.

What to submit

- Your source file (C++ or Python) implementing the four mechanisms.
- Program output reproducing: the admission decisions, the policing pass/drop sequence, the shaping release times and delays, and the WFQ finish numbers and order.
- The written answers: Part A (which guarantee the rejection protects), Part C (the shaping-versus-policing difference), and Part D (how WFQ prevents starvation).

Appendix: the interface each program must implement

- **Admission:** `admit(requests, C) -> decisions`, applying the Parekh-Gallagher test sequentially.
- **Policing:** `police(trace, r, b) -> per-packet pass or drop`.
- **Shaping:** `shape(trace, r, b) -> release time per packet`.

- **Scheduling:** `wfq(flows) -> finish numbers and output order`, and `fifo(flows) -> arrival order`, where each flow carries its weight and its packets.

Only the produced values are graded, not the types. The token bucket conforms a packet of size L when at least L tokens are present; policing drops the excess, shaping buffers it. The WFQ finish number is max(arrival, previous finish in the flow) plus length divided by weight.